smaller than 'A20' when compared alphabetically because the character "1" occurs earlier in the ASCII codepage than the character "2". A human in a hurry, however, might prefer to see 'A20' in a list before 'A100.

For use cases like this, `natsort()` produces the needed result. Note that `sort()` accepts the modifier flag of `SORT_NATURAL` to produce the same behavior.

Randomizing Arrays

The opposite of `sort()` and its brethren is `shuffle()`:

```
$arr = [1, 2, 3, 4];
shuffle($arr);

print_r($arr); // $arr is randomised, output will vary

Output:
Array
(
    [0] => 3
    [1] => 1
    [2] => 4
    [3] => 2
)
```

`shuffle()` generates a new set of randomized index values for the elements, then puts the array into that order. For that reason, `shuffle()` is not safe to use on associative arrays unless you want to lose all the keys. However, there is a way to shuffle an associative array and preserve key association by using shuffle on the output of `array_keys()`:

```
$car = [
    'make' => 'VW',
    'model' => 'Golf',
    'variant' => 'GTI 8v',
    'colour' => 'Oak Green',
];
```